

Протокол взаимодействия EU

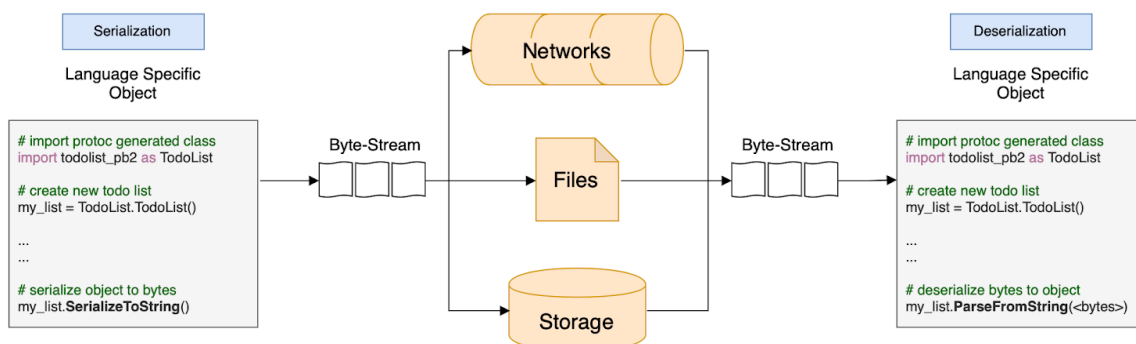
Общее описание

За основу протокола взаимодействия между SBC Raspberry (Linux) и платами DCB и ECB электронного блока EU взят **Protocol Buffers** (<https://protobuf.dev/overview/>). С его помощью решается задача сериализации и десериализации сообщений сложной структуры.

Для STM32 используется упрощенная реализация и компилятор Protocol Buffers - Nanopb

- <https://jpa.kapsi.fi/nanopb/>
- <https://github.com/nanopb/nanopb.git>

Структура сообщений описывается в специальном файле с расширением `.proto`. Затем компилятор `protoc` по данному файлу структуры сообщения создает необходимые файлы кода с соответствующими классами и структурами для интеграции в приложение, упрощающие работу с сообщениями

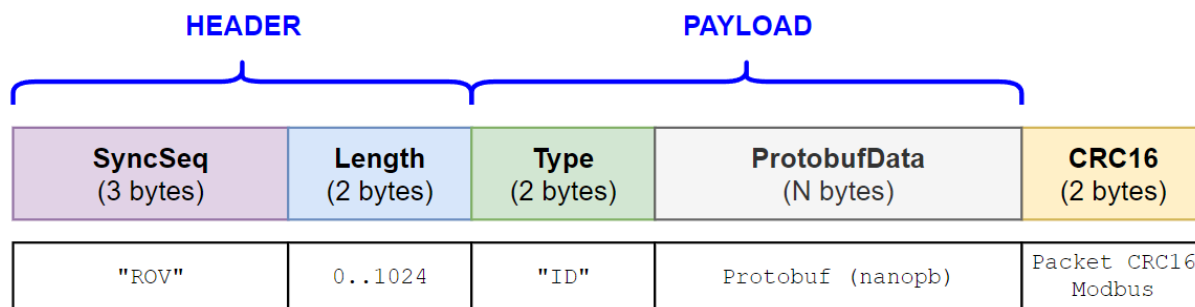


(<https://www.freecodecamp.org/news/googles-protocol-buffers-in-python/>)

Структура пакета

Так как Protocol Buffers переводит сообщения в бинарную строку, необходимо добавить к этому какие-то механизмы разбора сообщений. Предлагается следующая структура пакета:

```
[SyncSeq (напр., "ROV")][Length (2 байта)][Type][ProtobufData][CRC (2 байта)]
```



Порядок байт: little-endian

Таким образом, у каждого пакета есть заголовок - некая последовательность символом синхронизации SyncSeq. Затем следуют два байта длины полезных данных блока PAYLOAD (поле типа сообщения + сериализованные бинарные данные ProtobufData). Далее сами сериализованные данные ProtobufData и в конце два байта контрольной суммы пакета (HEADER + PAYLOAD).

Поле	Значение	Длина (байт)	Описание
SyncSeq	"ROV"	3	Последовательность синхронизации пакета
Length	0..1024	2	Кол-во байт блока полезных данных PAYLOAD
Type		2	Тип Protobuf сообщения. Необходим для правильной интерпретации структуры данных сообщения
ProtobufData		0..N	Бинарное сообщение в формате Protobuf
CRC	CRC-16 (Modbus)	2	Контрольная сумма пакета в формате CRC16 (Modbus)

CRC-16 (Modbus) — один из самых популярных вариантов для встраиваемых систем (STM32, Arduino) и Python.

Блок полезных данных PAYLOAD

Type

Тип сообщения Protobuf, переданной в PAYLOAD. Формат - символьный "ID" .
|Type|Description|

Настройка UART STM32

UART на стороне STM32 настроен следующим образом:

- Скорость: 115200
- Два канала DMA (tx/rx)
- Включено прерывание Idle Line Detection (в функции `void initUARTtoRx()`)
Передача данных tx/rx осуществляется с помощью DMA. Окончание приема пакета определяется по срабатыванию Idle Line прерывания.

Note

Прерывание свободной линии Idle Line UART возникает при включении МК после инициализации UART. Это побочная реакция, которая фильтруется в callback `idleLineDetectionCallBack()` путем проверки количества принятых байт. Если мы не запускали прием данных -> 0 принятых байт, значит срабатывание ложное

Алгоритм приема пакета

FreeRTOS запускает отдельную задачу по приему данных по протоколу и парсингу `StartCommandProcessingTask()` . В начале задачи происходит настройка UART.
В цикле:

1. Запрос на чтение байт по UART через DMA. Запрашивается максимальное количество байт (`DMA_RX_BUFFER_SIZE=512`), равное размеру приемного буфера, так как нам неизвестна длина пакета
2. Ожидание окончания приема, которое регистрируется или как Idle Line прерывания приемный буфер заполнен до максимального значения (освобождается семафор задачи)
3. После этого выполняется парсинг пакета, где пакет проверяется на соответствие формату, извлекается полезная нагрузка PAYLOAD
4. Проверка типа сообщения из PAYLOAD, вызов или передача управления тем функциям и задачам, которые соответствуют типу сообщения

Функционал работы с пакетом данных расположен в отдельном модуле `cmd_processing.h`, `cmd_processing.c`